



Arah Karya Sinergi

AKS · SIMPLY MORE.

PRODUCT REQUIREMENTS DOCUMENT

FALCON PRD

Spesifikasi teknis FALCON — kontrak byte telemetry, API contract, behavior per-komponen. Acuan build host-side (Simulator, Backend, Dashboard).

Architectural Precision in Digital Era.

Disiapkan oleh **Arah Karya Sinergi (AKS)**

× **NOZ (Tri Sumarno) — Telecom Security Research**

● Topik 4865 · FPGA
Juni 2026 · v1.0

Product Requirements Document (PRD)

FALCON — FPGA GTP-U DPI Engine & Telemetry Platform

FALCON · FPGA-Accelerated Live Core Observation Node

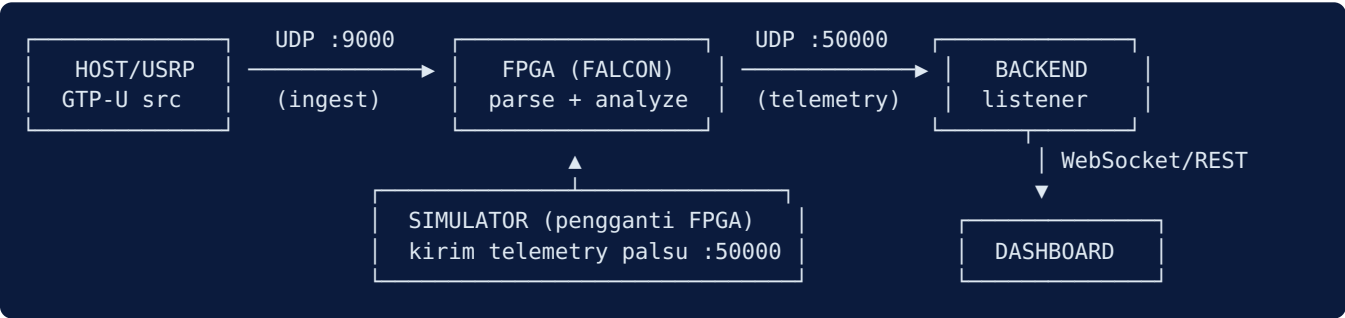
Dokumen	Product Requirements Document (PRD)
Project	FALCON
Versi	1.0 (draft)
Tanggal	Juni 2026
Disusun oleh	Arah Karya Sinergi (AKS)
Kolaborator	NOZ — Tri Sumarno (Telecom Security Research)
Referensi	BRD FALCON v1.0
Status	Draft — kontrak byte ditandai (<i>proposed</i> AKS) perlu konfirmasi NOZ

Catatan penting: Struktur byte telemetry di §5 adalah **proposal AKS** berdasarkan data parsial dari dokumen NOZ. Field & offset eksak **WAJIB dikonfirmasi NOZ** sebelum gateway final. Sampai dikonfirmasi, kontrak ini menjadi **acuan kerja** untuk Simulator + Backend (sisi host) agar pengembangan tidak ter-block.

1. Tujuan Dokumen

PRD ini menjabarkan **gimana FALCON bekerja** secara teknis: kontrak data antar-komponen, behavior tiap layer, API, dan acceptance criteria. PRD adalah **sumber kebenaran** untuk tiga komponen host-side yang dibangun lebih dulu (tanpa FPGA fisik): **Simulator, Backend, Dashboard**.

2. Arsitektur Komponen



Komponen	Peran	Fase Build	Butuh FPGA?
FPGA (FALCON gateway)	Parse GTP-U, hasilkan telemetry	Nanti (di NOZ)	✓ Ya
Simulator	Tiru output FPGA, kirim telemetry palsu	Sekarang	✗ Tidak

Komponen	Peran	Fase Build	Butuh FPGA?
Backend	Terima telemetry, parse, sajikan API	Sekarang	✗ Tidak
Dashboard	Visualisasi real-time	Sekarang	✗ Tidak

Simulator & Backend berbagi **kontrak byte yang sama** (\$5) → saat FPGA asli datang, Simulator dicabut, output FPGA langsung kompatibel.

3. Persona & User Stories

Persona	Kebutuhan	User Story
Analisis jaringan	Lihat siapa aktif & sesi apa	"Sebagai analis, saya ingin melihat jumlah IMSI aktif & per-sesi TEID real-time agar bisa memantau beban jaringan."
Security researcher (NOZ)	Deteksi event & anomali	"Sebagai peneliti, saya ingin notifikasi saat sesi baru dibuat agar bisa menganalisa pola signaling."
Operator/Ndoro	Bukti sistem jalan	"Sebagai owner, saya ingin dashboard live agar bisa mendemokan kapabilitas FALCON ke mitra."

4. Transport & Konvensi

Aspek	Nilai	Catatan
Ingest Host→FPGA	UDP dst-port 9000	Hardcoded komparator FPGA
Telemetry FPGA→Host	UDP dst-port 50000	Host listen di 50000 (<i>keputusan AKS, lihat BRD §8.1</i>)
Endianness	Big-endian (network byte order)	Konvensi telco/3GPP; konfirmasi NOZ
Encoding string/IMSI	ASCII digit, fixed-length, null-pad	IMSI maks 15 digit
Header tiap message	4 byte: <code>msg_type(1) · version(1) · length(2)</code>	Common header semua tipe (<i>proposed AKS</i>)

4.1 Common Telemetry Header (*proposed AKS*)

Setiap datagram telemetry diawali header 4 byte:

Offset	Field	Tipe	Keterangan
0	<code>msg_type</code>	uint8	0x01 / 0x02 / 0x03 / 0x04
1	<code>version</code>	uint8	0x01 (versi protokol telemetry)
2	<code>length</code>	uint16 BE	panjang payload setelah header (byte)

Header ini memudahkan Backend mem- `switch(msg_type)` & validasi panjang. Bila NOZ tidak memakai header, Backend menyesuaikan; struktur payload (\$5) tetap acuan.

5. Kontrak Byte Telemetry (*proposed AKS — konfirmasi NOZ*)

Ukuran payload mengacu pada angka di dokumen NOZ: 0x01=64B, 0x02=48B, 0x03=32B, 0x04=32B (sudah termasuk/diluar header → diasumsikan **payload** di luar 4-byte header).

5.1 TYPE 0x01 — Global Statistics (periodik, tiap 1 dtk)

Offset	Field	Tipe	Contoh	Keterangan
0	timestamp	uint32 BE	epoch detik	waktu sampel
4	total_imsi	uint32 BE	142	jumlah IMSI aktif
8	uplink_pps	uint32 BE	18000	paket/dtk uplink
12	downlink_pps	uint32 BE	—	paket/dtk downlink
16	active_teid	uint32 BE	—	sesi aktif
20	total_bytes	uint64 BE	—	akumulasi byte
28	drop_count	uint32 BE	—	paket drop
32..	reserved	bytes	—	padding s/d 64B

5.2 TYPE 0x02 — Per-TEID Statistics (per interval, per sesi)

Offset	Field	Tipe	Contoh	Keterangan
0	teid	uint32 BE	0xA1B2C3D4	Tunnel Endpoint ID
4	imsi	char[16]	"310410123456789"	ASCII, null-pad
20	qfi	uint8	7	QoS Flow ID
21	state	uint8	1=ACTIVE	status sesi
22	uplink_pkts	uint32 BE	—	paket UL sesi
26	downlink_pkts	uint32 BE	—	paket DL sesi
30..	reserved	bytes	—	padding s/d 48B

5.3 TYPE 0x03 — Event Notification (real-time)

Offset	Field	Tipe	Contoh	Keterangan
0	event_type	uint8	1=CreateSession	jenis event
1	direction	uint8	0=UL,1=DL	arah
2	teid	uint32 BE	0xAABBCCDD	sesi terkait
6	packet_len	uint16 BE	312	panjang paket pemicu
8	timestamp	uint32 BE	—	waktu event
12..	reserved	bytes	—	padding s/d 32B

5.4 TYPE 0x04 — Protocol Distribution (periodik)

Offset	Field	Tipe	Keterangan
0	<code>gtp_u_pct</code>	uint16 BE	% komposisi GTP-U (basis 100 atau 10000)
2	<code>gtp_c_pct</code>	uint16 BE	% GTP-C
4	<code>pfcp_pct</code>	uint16 BE	% PFCP
6	<code>bssgp_pct</code>	uint16 BE	% BSSGP
8	<code>other_pct</code>	uint16 BE	% lainnya
10..	<code>reserved</code>	bytes	padding s/d 32B

6. API Contract — Backend → Dashboard

Backend mengubah byte telemetry menjadi JSON dan menyajikan via WebSocket (push real-time) + REST (snapshot).

6.1 WebSocket — `ws://<host>:8080/ws`

Server push pesan JSON tiap kali telemetry diterima/diagregasi:

```
{
  "type": "global",
  "ts": 1718900000,
  "data": { "total_imsi": 142, "uplink_pps": 18000, "downlink_pps": 9500,
            "active_teid": 130, "drop_count": 0 }
}
```

```
{
  "type": "teid",
  "ts": 1718900000,
  "data": { "teid": "0xA1B2C3D4", "imsi": "310410123456789",
            "qfi": 7, "state": "ACTIVE", "ul_pkts": 1200, "dl_pkts": 980 }
}
```

```
{
  "type": "event",
  "ts": 1718900000,
  "data": { "event": "CreateSession", "teid": "0xAABBCCDD",
            "direction": "UL", "packet_len": 312 }
}
```

```
{
  "type": "protocol",
  "ts": 1718900000,
  "data": { "gtp_u": 78.2, "gtp_c": 9.1, "pfcp": 6.4, "bssgp": 3.0, "other": 3.3 }
}
```

6.2 REST Endpoints

Method	Path	Respons
GET	/api/health	{ "status": "ok", "uptime_s": N }
GET	/api/stats/global	snapshot global terakhir
GET	/api/stats/teid	array sesi aktif terakhir
GET	/api/events?limit=N	N event terbaru (ring buffer)
GET	/api/stats/protocol	distribusi protokol terakhir

7. Behavior per Komponen

7.1 Simulator (pengganti FPGA)

- Kirim datagram UDP ke `127.0.0.1:50000` dengan struktur byte §5.
- Jadwal: 0x01 tiap 1s · 0x04 tiap 1s · 0x02 per sesi tiap 2s · 0x03 event acak (Poisson).
- Data realistis: pool TEID/IMSI acak, pps berfluktuasi, kadang event CreateSession.
- Flag `--rate` untuk stress test.

7.2 Backend

- UDP listener async di `:50000` → baca header → `switch(msg_type)` → unpack payload (struct) → normalisasi JSON.
- Simpan state terakhir (global, protocol), tabel sesi TEID (TTL/expire), ring-buffer event (N terakhir).
- Push ke semua klien WebSocket; layani REST snapshot.
- Robust: abaikan datagram malformed, log warning, jangan crash.

7.3 Dashboard

- Connect WebSocket, render kartu KPI (IMSI aktif, pps, drop), tabel sesi TEID live, feed event, donut protokol.
- Tema **Hermes Dashboard** (navy/cyan, flat, mono untuk angka).
- Auto-reconnect bila WS putus; indikator status koneksi.

8. Acceptance Criteria

ID	Kriteria	Cara Verifikasi
AC-1	Simulator kirim 4 tipe message valid ke :50000	tcpdump/hexdump cek byte
AC-2	Backend parse semua tipe tanpa crash pada 100 msg/s	run + log, 0 error
AC-3	Backend malformed-safe	inject byte rusak → tetap jalan
AC-4	WebSocket push JSON sesuai §6.1	klien uji terima 4 tipe
AC-5	REST endpoints balas snapshot benar	curl tiap endpoint
AC-6	Dashboard tampil real-time, update ≤ 1 dtk	mata + DevTools WS frames
AC-7	Dashboard auto-reconnect	matikan backend → nyalakan → pulih

ID	Kriteria	Cara Verifikasi
AC-8	End-to-end: simulator→backend→dashboard live	demo penuh

9. Stack Teknis (host-side)

Layer	Pilihan	Alasan
Simulator	Python (<code>socket</code> , <code>struct</code>)	cepat, mudah ubah
Backend	Python <code>asyncio</code> + <code>websockets</code> + HTTP (aiohttp/FastAPI)	async UDP+WS+REST satu proses
Dashboard	HTML + JS vanilla (no framework berat)	ringan, tema Hermes Dashboard inline
Serialisasi	<code>struct</code> (byte) ↔ JSON	sesuai kontrak §5/§6

RPi5-friendly: footprint kecil, tanpa build berat. Bisa jalan lokal untuk demo.

10. Open Questions (untuk NOZ)

#	Pertanyaan	Dampak
Q1	Apakah ada common header 4-byte, atau payload langsung?	Parsing offset
Q2	Endianness big-endian dikonfirmasi?	Unpack benar/salah
Q3	Field eksak & offset tiap tipe (terutama 0x01 64B isi apa saja)?	Kontrak final
Q4	Basis persentase 0x04 (skala 100 atau 10000)?	Tampilan %
Q5	Encoding IMSI (ASCII digit vs BCD/TBCD 3GPP)?	Decode IMSI
Q6	Interval pengiriman 0x02 & trigger 0x03 pasti?	Tuning simulator

Jawaban NOZ → kontrak §4–5 difinalkan, lalu Simulator + Backend di-patch. Karena Backend pakai header `length` , perubahan field internal berdampak minimal.

11. Definition of Done (Fase host-side / A)

- [] Simulator, Backend, Dashboard jalan bersama, demo end-to-end live.
- [] Semua AC-1..8 lulus.
- [] Kontrak byte terdokumentasi di kode (satu modul shared) = mirror PRD §5.
- [] README cara menjalankan.
- [] Siap colok FPGA asli: cukup matikan Simulator.

Dokumen ini adalah PRD (fokus: gimana produknya jalan). Kontrak byte §5 = proposal AKS, akan difinalkan setelah input NOZ.

Disiapkan oleh Arah Karya Sinergi (AKS) — kolaborasi NOZ.